

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №4

Интегрирование

(по дисциплине «Вычислительная математика»)

Выполнил:

Лабин Макар Андреевич,
ВЫЧМ 3.2, Р3231, 466449.

Проверила:

Перл Ольга Вячеславовна

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Постановка задач	1
2	Выполнение работы	2
2.1	Описание метода	2
2.1.1	Определение квадратурных формул	2
2.1.2	Степень точности квадратурной формулы	2
2.1.3	Формула трапеций	2
2.1.4	Обработка точек разрыва	3
2.2	Блок-схема численного метода	4
2.3	Листинг численного метода	7
2.4	Результаты работы программы	12
3	Заключение	14

1 Постановка задач

Дана функция $f(x)$. Необходимо вычислить определённый интеграл от заданной функции на интервале $[a, b]$ с заданным параметром точности ε — максимальной разницей между двумя итерациями (*разбиениями на отрезки*).

В варианте №2 вычисление осуществляется *методом трапеций*. При реализации алгоритма необходимо учитывать следующие условия:

- если у функции есть разрыв второго рода, скачок, или она не определена на какой-либо части интервала $[a, b]$, то следует установить переменные `error_message` и `hasDiscontinuity`, а также вывести сообщение: *"Integrated function has discontinuity or does not defined in current interval"*;
- если есть устранимый разрыв первого рода, вычисление интеграла должно быть выполнено корректно;
- если выполнено условие $a > b$, значение интеграла должно иметь отрицательный знак.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Разработка итерационного алгоритма численного интегрирования методом трапеций с контролем точности по заданному параметру ε .
2. Реализация механизма диагностики особенностей подынтегральной функции и обработки исключительных ситуаций согласно требованиям.
3. Организация ввода данных в формате $a \ b \ f \ \varepsilon$ и формирование вывода в виде вычисленного значения интеграла I .

2 Выполнение работы

2.1 Описание метода

2.1.1 Определение квадратурных формул

Рассмотрим задачу численного вычисления определённого интеграла от функции $f(x)$ на отрезке $[a, b]$:

$$I = \int_a^b f(x) dx.$$

Квадратурной формулой (или *аппроксимирующей квадратурой*) называется выражение вида

$$Q_n[f] = \sum_{i=0}^n w_i f(x_i),$$

где $\{x_i\}_{i=0}^n \subset [a, b]$ — узлы квадратуры, а $\{w_i\}_{i=0}^n$ — весовые коэффициенты. Квадратурная формула аппроксимирует интеграл I конечной линейной комбинацией значений подынтегральной функции.

Погрешность квадратуры определяется как разность следующего вида:

$$E_n[f] = I - Q_n[f]$$

2.1.2 Степень точности квадратурной формулы

Алгебраической степенью точности квадратурной формулы называется наибольшее целое неотрицательное число m , такое что формула точна для всех алгебраических многочленов степени не выше m :

$$\forall k = \overline{0, m} \quad E_n[x^k] = 0, \quad \text{но} \quad E_n[x^{m+1}] \neq 0$$

Иными словами, если подынтегральная функция является многочленом $P_m(x)$ степени m , то квадратурная формула даёт точное значение интеграла.

2.1.3 Формула трапеций

Рассмотрим отрезок $[a, b]$ и построим интерполяционный многочлен Лагранжа первой степени на узлах $\{a, b\}$:

$$L_1(x) = \frac{x-b}{a-b} f(a) + \frac{x-a}{b-a} f(b)$$

Определим остаточный член $R_1(x)$ согласно теореме об остаточном члене интерполяционной формулы Лагранжа:

$$f(x) = L_1(x) + R_1(x), \quad R_1(x) = \frac{f''(\xi(x))}{2} (x-a)(x-b), \quad \xi(x) \in (a, b)$$

Проинтегрируем обе части равенства по отрезку $[a, b]$:

$$\int_a^b f(x) dx = \int_a^b L_1(x) dx + \int_a^b R_1(x) dx$$

Вычислим интеграл от интерполяционного многочлена почленно:

$$\begin{aligned}\int_a^b L_1(x) dx &= f(a) \int_a^b \frac{x-b}{a-b} dx + f(b) \int_a^b \frac{x-a}{b-a} dx \\ &= \frac{f(a)}{a-b} \left[\frac{(x-b)^2}{2} \right]_a^b + \frac{f(b)}{b-a} \left[\frac{(x-a)^2}{2} \right]_a^b \\ &= \frac{f(a)}{a-b} \left(0 - \frac{(a-b)^2}{2} \right) + \frac{f(b)}{b-a} \left(\frac{(b-a)^2}{2} - 0 \right) \\ &= \frac{b-a}{2} f(a) + \frac{b-a}{2} f(b) = \frac{b-a}{2} (f(a) + f(b))\end{aligned}$$

Для оценки погрешности проинтегрируем остаточный член. Поскольку $(x-a)(x-b) \leq 0$ на $[a, b]$, а f'' непрерывна, по интегральной теореме о среднем:

$$\int_a^b \frac{f''(\xi(x))}{2} (x-a)(x-b) dx = \frac{f''(\xi)}{2} \int_a^b (x-a)(x-b) dx, \quad \xi \in (a, b)$$

Вычислим интеграл от квадратного многочлена:

$$\int_a^b (x-a)(x-b) dx = \int_a^b (x^2 - (a+b)x + ab) dx = -\frac{(b-a)^3}{6}$$

Подставляя результат, получаем остаточный член формулы трапеций:

$$\int_a^b R_1(x) dx = -\frac{(b-a)^3}{12} f''(\xi)$$

Окончательно, полная формула трапеций записывается в виде:

$$\int_a^b f(x) dx = \frac{f(a) + f(b)}{2} \cdot h - \frac{h^3}{12} f''(\xi), \quad \xi \in (a, b), \quad h = b - a$$

Можно заметить, что алгебраическая степень точности метода трапеций равна $m = 1$. Это следует из погрешности квадратуры: если f — многочлен первой степени, то $R_1(x) \equiv 0$.

2.1.4 Обработка точек разрыва

Если функция $f(x)$ имеет разрывы на отрезке интегрирования, квадратурная формула может давать некорректный результат или приводить к исключительным ситуациям (*деление на ноль, переполнение*).

1. **Устранимый разрыв или скачок.** Интеграл Римана существует, и алгоритм должен заменить значение $f(c)$ на одно из значений $f(c - \varepsilon(h))$, $f(c + \varepsilon(h))$, где $\varepsilon(h)$ — некоторая малая величина.
2. **Бесконечный разрыв.** Интеграл Римана не существует, и алгоритм должен прервать вычисления, уведомив пользователя об ошибке.

Таким образом, в реализации метода трапеций мы должны проанализировать подынтегральную функцию на наличие особенностей и адаптировать разбиение интервала интегрирования в зависимости от типа разрыва.

2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- f_n — номер подынтегральной функции; целое число.
- a, b — границы интервала интегрирования; вещественные числа двойной точности.
- ε — точность вычислений, то есть максимально допустимая разность между двумя последовательными итерациями; вещественное число двойной точности.
- M — максимально допустимое число итераций; целое число.
- $\mathcal{D}$ — метаданные о расположении и типах разрывов функции (*точки первого/второго рода, интервалы неопределённости*).

На выходе метода ожидается вещественное число двойной точности — вычисленное значение интеграла.

Остановка итерационного процесса происходит при достижении заданной точности ε ($|I_k - I_{k-1}| < \varepsilon$) или при превышении лимита итераций M .

Далее приводятся блок-схемы, реализующие численный метод.

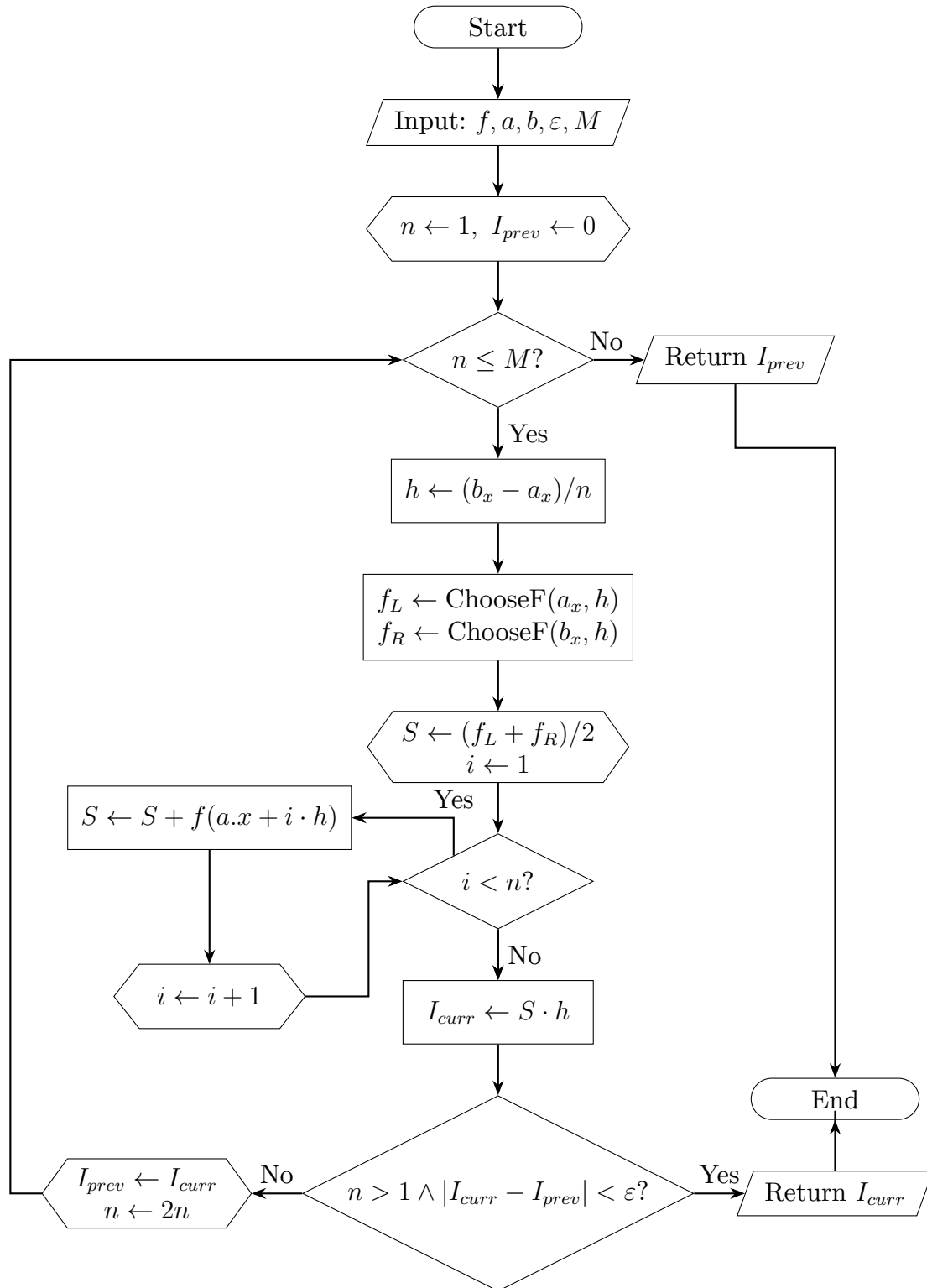


Схема 1 — Блок-схема безопасного вычисления формулы трапеций.

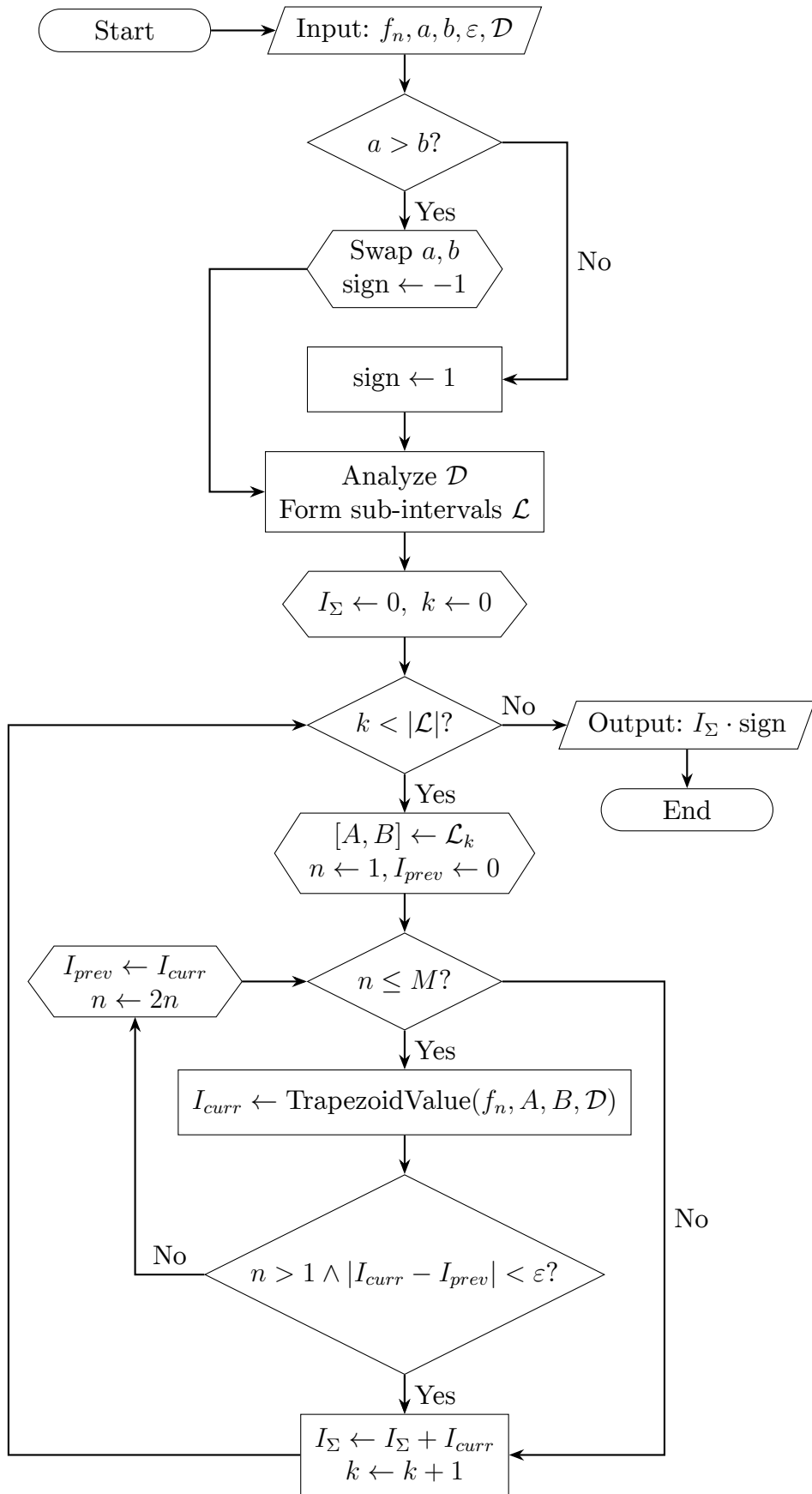


Схема 2 — Блок-схема адаптивного метода трапеций.

2.3 Листинг численного метода

Для реализации блок-схем воспользуемся языком программирования C++.

```
1 #ifndef INTEGRATION_H
2 #define INTEGRATION_H
3
4 #include <limits>
5
6 #include "preamble.h"
7 #include "unordered_map"
8 #include "vector"
9
10 struct DiscontinuityDot {
11     double x;
12     // 0 - continuous
13     // 1 - first order
14     // 2 - second order
15     std::size_t order;
16 };
17
18 struct DiscontinuityPlace {
19     bool is_interval;
20     DiscontinuityDot left;
21     DiscontinuityDot right;
22 };
23
24 struct FunctionMeta {
25     std::function<double(double)> f;
26     std::vector<DiscontinuityPlace> discontinuity_places;
27 };
28
29 struct IntegrationResult {
30     double result;
31     bool has_result;
32     bool has_discontinuity;
33     std::string message;
34 };
35
36 class IntegrateFunctions {
37 public:
38     IntegrateFunctions(double epsilon)
39         : IntegrateFunctions(epsilon, 1e6, std::unordered_map<std::
              ::size_t, FunctionMeta>{}) {
40     }
41
42     IntegrateFunctions(
43         double epsilon,
44         std::size_t max_iterations,
45         std::unordered_map<std::size_t, FunctionMeta> funcs
46     )
47         : _funcs(funcs), _epsilon(epsilon), _max_iterations(
```

```

max_iterations) {
48 }
49
50 void AddFunction(const std::size_t number, const FunctionMeta&
    func) {
51     this->_funcs[number] = func;
52 }
53
54 IntegrationResult Integrate(std::size_t number, double a,
    double b) {
55     double result_sign = 1.0;
56     if (b < a) {
57         std::swap(a, b);
58         result_sign = -1.0;
59     }
60     if (this->_funcs.count(number) == 0U) {
61         throw std::invalid_argument("no integration functions
            registered under specified number");
62     }
63     double result = 0.0;
64     FunctionMeta meta = this->_funcs[number];
65     std::vector<DiscontinuityDot> split_points = {
66         {a, 0}
67     };
68
69     for (const auto& place : meta.discontinuity_places) {
70         if (place.is_interval) {
71             if (place.left.x < b && a < place.right.x) {
72                 return {
73                     0.0,
74                     false,
75                     true,
76                     "Integrated function has discontinuity or does not
                        defined in current interval"};
77             }
78         } else {
79             double dx = place.left.x;
80             if (dx >= a && dx <= b) {
81                 if (place.left.order == 2) {
82                     return {
83                         0.0,
84                         false,
85                         true,
86                         "Integrated function has discontinuity or does
                            not defined in current interval"};
87                 }
88                 if (dx != a && dx != b) {
89                     split_points.push_back({dx, place.left.order});
90                 }
91             }
92         }

```

```

93     }
94     split_points.push_back({b, 0});
95
96     double total_result = 0.0;
97     for (std::size_t i = 0; i < split_points.size() - 1; ++i) {
98         DiscontinuityDot sub_a = split_points[i];
99         DiscontinuityDot sub_b = split_points[i + 1];
100         total_result += _IntegrateSubInterval(meta.f, sub_a, sub_b
101             );
102     }
103
104     return {
105         total_result * result_sign,
106         true,
107         false,
108     };
109 }
110
111 private:
112 double _IntegrateSubInterval(
113     const std::function<double(double)>& f, const
114     DiscontinuityDot& a, const DiscontinuityDot& b
115 ) {
116     auto safe_eval = [&](double x, double h_step, bool is_left,
117         bool is_continuous) -> double {
118         if (!is_continuous) {
119             double eps = std::max(1e-12, h_step * 0.01);
120             if (is_left) {
121                 double v_r = f(x + eps);
122                 bool bad_r = std::isnan(v_r) || std::isinf(v_r);
123                 if (!bad_r) {
124                     return v_r;
125                 }
126             }
127
128             double v_l = f(x - eps);
129             bool bad_l = std::isnan(v_l) || std::isinf(v_l);
130             if (!bad_l) {
131                 return v_l;
132             }
133             return 0.0;
134         }
135         return f(x);
136     };
137
138     double prev_res = 0.0;
139     std::size_t n = 1;
140
141     while (n <= this->_max_iterations) {
142         double h = (b.x - a.x) / n;
143         double current_res =
144             0.5 * (safe_eval(a.x, h, true, a.order == 0) +

```

```

141         safe_eval(b.x, h, false, b.order == 0));
142     for (std::size_t i = 1; i < n; i++) {
143         current_res += f(a.x + i * h);
144     }
145     current_res *= h;
146
147     if (n > 1 && std::abs(current_res - prev_res) < this->
148         _epsilon) {
149         return current_res;
150     }
151     prev_res = current_res;
152     n *= 2;
153 }
154
155 std::unordered_map<std::size_t, FunctionMeta> _funcs;
156 const std::size_t _max_iterations;
157 const double _epsilon;
158 };
159
160 IntegrateFunctions InitMain(double epsilon) {
161     IntegrateFunctions integrate = IntegrateFunctions(epsilon);
162     integrate.AddFunction(
163         1,
164         FunctionMeta{
165             get_function(1),
166             {
167                 DiscontinuityPlace{
168                     false,
169                     {
170                         0.0,
171                         2,
172                     },
173                 },
174             }
175         );
176     integrate.AddFunction(
177         2,
178         FunctionMeta{
179             get_function(2),
180             {
181                 DiscontinuityPlace{
182                     false,
183                     {
184                         0.0,
185                         1,
186                     },
187                 },
188             }
189         );

```

```

190     integrate.AddFunction(
191         3,
192         FunctionMeta{
193             get_function(3),
194             {},
195         }
196     );
197     integrate.AddFunction(
198         4,
199         FunctionMeta{
200             get_function(4),
201             {},
202         }
203     );
204     integrate.AddFunction(
205         5,
206         FunctionMeta{
207             get_function(5),
208             {
209                 DiscontinuityPlace{
210                     true,
211                     {
212                         -std::numeric_limits<double>::infinity(),
213                         0,
214                     },
215                     {
216                         0.0,
217                         2,
218                     },
219                 }, },
220             }
221     );
222     return integrate;
223 }
224
225 double calculate_integral(double a, double b, int f, double
226     epsilon) {
227     IntegrateFunctions integrate = InitMain(epsilon);
228     IntegrationResult result = integrate.Integrate(static_cast<std
229         ::size_t>(f), a, b);
230     has_discontinuity = result.has_discontinuity;
231     if (!result.has_result) {
232         error_message = result.message;
233     }
234     return result.result;
235 }
236
237 #endif

```

Листинг 1 — Файл integration.hpp

```

1 #include "lib/iterations.hpp"

```

```

2
3 #include <iostream>
4 #include <unordered_map>
5
6 int main() {
7     SimpleIterationsSystem obj = InitSystem();
8
9     std::unordered_map<std::size_t, std::vector<double>> initial_x
10         = {
11         {1, { 0.0, 0.0 }},
12         {2, { -1.9, 2.1 }},
13         {3, { 0.0, 0.0, 0.0 }},
14         {4, { 0.8, 0.6 }},
15         {5, { 0.4, 0.2 }},
16     };
17     for (std::size_t i = 1; i <= 5; i++) {
18         std::cout << "--- Solution for system #" << i << ":\n";
19         ResultInfo rawInfo = obj.SolveSystem(i, initial_x[i]);
20         std::vector<double> result = rawInfo.solution;
21         std::vector<double> residues = CaluclateResidues(i, result);
22         for (std::size_t j = 0; j < result.size(); j++) {
23             std::cout << "x[" << j << "]: " << result[j] << "\t\t" <<
24                 "res[" << j << "]: " << residues[j] << "\n";
25         }
26         double mse = 0.0;
27         for (const auto& res : residues) {
28             mse += res * res;
29         }
30         mse /= residues.size();
31         std::cout << "MSE: " << mse << "\n";
32         std::cout << "Total iterations: " << rawInfo.
33             iterations_count << "\n";
34     }
35 }

```

Листинг 2 — Файл main.cpp

2.4 Результаты работы программы

Для демонстрации работы метода и тестирования программы рассмотрим пять функций с разными особенностями.

1. *Тригонометрическая система:*

$$\begin{cases} \sin(x_1) = 0 \\ \frac{x_1 x_2}{2} = 0 \end{cases} \implies \begin{cases} x_1 = x_1 - \sin(x_1) \\ x_2 = x_2 - \frac{x_1 x_2}{2} \end{cases}$$

2. *Алгебраическая система высоких степеней:*

$$\begin{cases} x_1^2 x_2^2 - 3x_1^3 - 6x_2^3 + 8 = 0 \\ x_1^4 - 9x_2 + 2 = 0 \end{cases} \implies \begin{cases} x_1 = -\sqrt[4]{9x_2 - 2} \\ x_2 = \sqrt[3]{\frac{x_1^2 x_2^2 - 3x_1^3 + 8}{6}} \end{cases}$$

3. Алгебраическая система:

$$\begin{cases} x_1 + x_1^2 - 2x_2x_3 - 0.1 = 0 \\ x_2 + x_2^2 + 3x_1x_3 + 0.2 = 0 \\ x_3 + x_3^2 + 2x_1x_2 - 0.3 = 0 \end{cases} \implies \begin{cases} x_1 = 0.1 - x_1^2 + 2x_2x_3 \\ x_2 = -0.2 - x_2^2 - 3x_1x_3 \\ x_3 = 0.3 - x_3^2 - 2x_1x_2 \end{cases}$$

4. Смешанная алгебраическая система:

$$\begin{cases} x_1 - \sqrt{1 - x_2^2} = 0 \\ x_2 - x_1^3 = 0 \end{cases} \implies \begin{cases} x_1 = \sqrt[3]{x_2} \\ x_2 = \sqrt{1 - x_1^2} \end{cases}$$

5. Трансцендентная система:

$$\begin{cases} x_1 + e^{x_1} - 2 = 0 \\ x_2 + \sin(x_2) - x_1 = 0 \end{cases} \implies \begin{cases} x_1 = \ln(2 - x_1) \\ x_2 = x_1 - \sin(x_2) \end{cases}$$

Ниже приведены результаты решения указанных систем методом простых итераций с заданной точностью $\varepsilon = 10^{-5}$.

№	$\vec{x}^{(0)}$	$\vec{x}^*$	σ^2	Итерации
1	$(0.0, 0.0)^T$	$(-0.0, -0.0)^T$	0	1
2	$(-1.9, 2.1)^T$	$(-2.0, 2.0)^T$	3.76559^{-12}	22
3	$(0.0, 0.0, 0.0)^T$	$(-0.0179359, -0.249651, 0.235557)^T$	1.79298^{-15}	29
4	$(0.8, 0.6)^T$	$(0.826031, 0.563624)^T$	4.43422^{-15}	79
5	$(0.4, 0.2)^T$	$(0.442854, 0.222341)^T$	4.63827^{-15}	584

Таблица 1 — Сводная таблица результатов тестирования.

3 Заключение

После выполнения лабораторной работы у меня появились ответы на следующие вопросы.

погрешность квадратурных формул интегрирования

следует анализировать максимальную величину шага разбиения, либо количество разбиений (особенно для методов с динамическим, а не постоянным шагом)

сравнить с методом Симпсона (методом парабол)

Почему степень точности квадратурной формулы определяется через многочлены? По аппроксимационной теореме Вейерштрасса, любую непрерывную на отрезке функцию можно сколь угодно точно приблизить алгебраическим многочленом. Следовательно, если квадратурная формула точно интегрирует многочлены высокой степени, то для гладких функций её погрешность будет определяться лишь погрешностью аппроксимации.

То есть степень точности описывает порядок малости погрешности метода при стремлении шага разбиения к нулю.

Почему используемый критерий останова может быть неточным?

В работе мы допустили, что метод сходится быстро. Если же сходимость «медленная» ($q \rightarrow 1$), то разность двух соседних приближений может быть очень малой, в то время как само решение всё ещё находится далеко от текущей точки. В таких ситуациях использование упрощенного критерия без учета q может привести к преждевременной остановке алгоритма до достижения требуемой точности.

Почему ряд тестов не получилось пройти на платформе Code&Test?

Полагаю, что причина кроется в высокой чувствительности к начальным условиям и виду эквивалентной системы. Если для тестовой системы спектральный радиус матрицы Якоби $q \geq 1$ в окрестности корня, то итерационный процесс расходится.

Какая асимптотическая сложность у алгоритма? Временная сложность метода простых итераций составляет $\mathcal{O}(M \cdot n)$, где M — количество затраченных итераций, а n — размерность системы. Как следует из блок-схемы, на каждой итерации происходит однократное вычисление вектора-функции $\Phi(x)$ и расчет нормы невязки за линейное время $\mathcal{O}(n)$. Здесь мы допускаем, что расчёт значения отображения Φ занимает константное время.

Пространственную сложность можно оценить $\mathcal{O}(n)$, поскольку алгоритм требует выделения памяти лишь под два вектора размерности n , что эффективнее прямых методов, требующих хранения квадратных матриц.

В чём преимущество метода простых итераций? Если сравнивать с методом Ньютона, главное преимущество — вычислительная простота одного шага. Метод простых итераций не требует вычисления элементов матрицы Якоби и решения системы линейных алгебраических уравнений на каждой итерации. Если сравнивать с методом градиентного спуска, то простому

итерационному процессу не нужно подбирать какие-либо гиперпараметры (в частности, шаг обучения).

Какие недостатки обнаружены у метода простых итераций? Главный недостаток — линейная скорость сходимости, которая существенно уступает квадратичной сходимости метода Ньютона.

Кроме того, метод требует трудоемкого предварительного этапа: ручного преобразования исходной системы $F(x) = 0$ к эквивалентному виду $x = \Phi(x)$. При этом преобразование должно быть выполнено так, чтобы гарантировалось выполнение достаточного условия сходимости.

Конечно, есть эмпирические модификации метода простых итераций, которые позволяют формировать преобразование Φ автоматически. Однако их рассмотрение выходит за рамки лабораторной работы и лекционного материала.

Почему число итераций у трансцендентной системы резко возросло?

В разделе с условиями сходимости метода было сказано, что её скорость линейна и зависит от нормы матрицы Якоби. Увеличение количества итераций лишь говорит о том, что q оказалось очень близко к единице. То есть каждый последующий шаг лишь незначительно приближает нас к ответу, и алгоритму требуется большое число итераций для достижения требуемой точности.